

Assignment 2

COMP4528

Xinni Song

U7439250

TALKING WITH HAOYU SONG U7439298

Task-1: Harris Corner Detector. (5 marks)

1. Read and understand the corner detection code in the file "harris.py".
2. Complete the missing parts, rewrite them to "harris.py" as a python script (or a Jupyter Notebook file), and design an appropriate function signature (1 mark).

There are two missing parts, the first one is "Compute the Harris Cornerness".

I design a function, named `Harris_cornerness`, it implements the main process of the Harris corner detection algorithm. The input is a grayscale image, and the output is the Harris corner response function image of each pixel. In this function, first use the Gaussian filter to smooth the input image, then calculate the gradient value of each pixel, then calculate the Harris corner response function of each pixel according to these gradient values, and finally output the corner Point response function graph. The following is the function signature of `Harris_cornerness`.

```
def Harris_cornerness(bw: np.ndarray, sigma: float, k: float, dx: np.ndarray, dy: np.ndarray) -> np.ndarray:
```

Calculates Harris corner response function for a grayscale image.

Args:

`bw (numpy.ndarray)`: The grayscale input image.

`sigma (float)`: The standard deviation of the Gaussian filter.

`k (float)`: Empirical constant in the Harris corner response function.

`dx (numpy.ndarray)`: Derivative kernel in x-direction.

`dy (numpy.ndarray)`: Derivative kernel in y-direction.

Returns:

`numpy.ndarray`: The Harris corner response function.

The second missing part is "Perform non-maximum suppression".

This function performs non-maximum suppression on a given corner response image to return a list of corner points that are the local maxima within a window. It takes as input the corner response image, the size of the window, and a threshold value. For each pixel in the image, a window is created around it, and the pixel is added to the corner list if it is the maximum within the window and its value is greater than or equal to the threshold times the maximum value in the entire image. The function returns a numpy array of corner points. The following is the function signature of `non_max_suppression`.

```
def non_max_suppression(image: np.ndarray, window_size: int, thresh: float) -> np.ndarray:
```

Apply non-maximum suppression to the corner response image to obtain the corner coordinates.

Args:

- `image`: 2D array representing the corner response image.

- `window_size`: integer representing the size of the window to find local maximum values.

- `thresh`: float representing the threshold value to filter out weak corners.

Returns:

- `corners`: 2D array of (x, y) coordinates representing the detected corner points.

3. Comment on line #53 (corresponding to line #13 in “harris.m”) and every line of your solution after line #60 (0.5 mark) in your report. Specifically, you need to provide short comments on your code, which should make your code readable. Please make sure you include the comments in your report to get marks.

#Code explanation of line #53:

```
1. g = fspecial((max(1, np.floor(3 * sigma) * 2 + 1), max(1, n
    p.floor(3 * sigma) * 2 + 1)), sigma)
```

Explanation:

This line of code defines a Gaussian filter `g`, which is used to perform Gaussian filtering on the input image. The function `fspecial()` is used to generate a special filter, the filter here is a Gaussian filter, and its generation method is controlled by the standard deviation `sigma` and the filter size. The size of the filter is determined by `np.floor(3 * sigma) * 2 + 1`, while the standard deviation `sigma` determines the shape of the Gaussian function. By applying Gaussian filtering to the image, the noise can be smoothed while preserving the edge and detail information in the image.

#Code explanation of line #60:

```
1. #sigma is the standard deviation of Gaussian filtering, thr
   esh is the threshold of the corner response function,
2. #k is the response function coefficient, and block_size is
   the window size used to calculate the corner response funct
   ion.
3. #The ksize parameter in this function represents the size o
   f the Sobel operator kernel used for calculating gradients
   in the x and y directions.
4. sigma = 2
5. thresh = 0.01
6. k = 0.04
7. block_size = 5
8. ksize = 3
9. #Define two convolution kernels dx and dy, which are used t
   o calculate the gradient value of pixels in the image.
10. #dx is the convolution kernel for derivation in the horizon
    tal direction, and dy is the convolution kernel for derivat
    ion in the vertical direction.
11. #They can be used to calculate the gradient values in the x
    and y directions of each pixel in the image.
12. dx = np.array([[-1, 0, 1], [-1, 0, 1], [-1, 0, 1]])
13. dy = dx.transpose()
```

Explanation

The code here is to set some parameters that will be used later, where `sigma` is the standard deviation of Gaussian filtering, `thresh` is the threshold of the corner response function, `k` is the response function coefficient, and `block_size` is the window size used when calculating the corner response function. The `ksize` parameter in this function represents the size of the Sobel operator kernel used for calculating gradients in the x

and y directions. The Sobel operator is used to compute the image gradients, which are then used to calculate the Harris response function. `dx` and `dy` are convolution kernels deriving in the horizontal and vertical directions, respectively, and are used to calculate the gradient values in the x and y directions of each pixel in the image. These variables and the convolution kernel will be used in the Harris corner detection function later.

Harris_cornerness

1. Overview:

```
2. def Harris_cornerness(bw, sigma, k, dx, dy):
3.     #Use the fspecial function defined previously to generate a Gaussian filter g.
4.     g = fspecial((max(1, np.floor(3 * sigma) * 2 + 1), max(1, np.floor(3 * sigma) * 2 + 1)), sigma)
5.     #Compute the gradient images Ix and Iy of the input image bw in the x and y directions
6.     #using the convolution operation with the kernels dx and dy.
7.     Ix = conv2(bw, dx)
8.     Iy = conv2(bw, dy)
9.     #Compute the three values Iy2, Ix2, and Ixy using the Harris corner detection algorithm formula,
10.    #which will be used to calculate the corner response function.
11.    Iy2 = conv2(np.power(Iy, 2), g)
12.    Ix2 = conv2(np.power(Ix, 2), g)
13.    Ixy = conv2(Ix * Iy, g)
14.    #calculate R
15.    #k is empirical constant, k = 0.01-0.1. We have set k as 0.04 above
16.    #det(M) = λ1λ2
17.    #trace(M) = λ1 + λ2
18.    #λ1 and λ2 are the eigenvalues of M
19.    det_M = Ix2 * Iy2 - np.power(Ixy, 2)
20.    trace_M = Ix2 + Iy2
21.    R = det_M - k * np.power(trace_M, 2)
22.    return R
```

2. Detail Explanation:

- 1) Use the `fspecial` function defined previously to generate a Gaussian filter `g`.

```
g = fspecial((max(1, np.floor(3 * sigma) * 2 + 1), max(1, np.floor(3 * sigma) * 2 + 1)), sigma)
```

- 2) Compute the gradient images `Ix` and `Iy` of the input image `bw` in the x and y directions using the convolution operation with the kernels `dx` and `dy`.

```
#Compute the gradient images Ix and Iy of the input image bw in the x and y directions
```

```

#using the convolution operation with the kernels dx and
dy.
Ix = conv2(bw, dx)
Iy = conv2(bw, dy)

```

- 3) Compute the three values I_y^2 , I_x^2 , and I_{xy} using the Harris corner detection algorithm formula, which will be used to calculate the corner response function.

```

#Compute the three values Iy2, Ix2, and Ixy using the Ha
rris corner detection algorithm formula,
#which will be used to calculate the corner response fun
ction.
Iy2 = conv2(np.power(Iy, 2), g)
Ix2 = conv2(np.power(Ix, 2), g)
Ixy = conv2(Ix * Iy, g)

```

- 4) Calculate the Harris corner response function R for each pixel using the formula $R = \det_M - k * \text{trace_M}^2$, where k is an empirical constant set to 0.04, $\det_M$ is the determinant of the matrix M , and trace_M is the trace of M . Calculated as follows,

$$\det(M) = \lambda_1 \lambda_2$$

$$\text{trace}(M) = \lambda_1 + \lambda_2$$

But we do not need get the specific lambd value, the reason is show as follow.

- Corner strength $R = \det(M) - k \text{Tr}(M)^2$
- Let λ_1 and λ_2 be the two eigenvalues.
We do not have to calculate them!
- Use trace and determinant:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \quad \begin{aligned} \det(\mathbf{A}) &= a_{11}a_{22} - a_{12}a_{21} \\ \text{tr}(\mathbf{A}) &= a_{11} + a_{22} \end{aligned}$$

Matrix M is as follow,

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

Therefore, we can get $\det M$ and $\text{trace} M$ without calculating the eigenvalues of the M matrix, as follows

$$\det M \equiv I_x^2 \cdot I_y^2 - (I_x \cdot I_y)^2$$

$$\text{trace} M \equiv I_x^2 + I_y^2$$

```

det_M = Ix2 * Iy2 - np.power(Ixy, 2)

```

```
trace_M = Ix2 + Iy2
```

Get R use the formula as follow:

$$R = \det(M) - k \cdot \text{trace}(M)^2$$

```
R = det_M - k * np.power(trace_M, 2)
```

5) Return the computed Harris corner response function R.

non_max_suppression

1. Overview

```
2. def non_max_suppression(image, window_size, thresh):
3.     #h and w represent the height and width of the corner response function image respectively
4.     h, w = image.shape
5.     #half of window size, used to find the Pixel-centered window
6.     offset = window_size // 2
7.     corner_list = []
8.     # Double cycle through the pixels of the entire image, where window represents the local window around the current pixel;
9.     #By comparing the value of the current pixel point with the maximum value in the surrounding window,
10.    #determine whether the current pixel point is greater than thresh, if so, add its coordinates to the corner_list
11.    for y in range(offset, h - offset):
12.        for x in range(offset, w - offset):
13.            window = image[y-offset:y+offset+1, x-offset:x+offset+1]
14.            if image[y, x] == np.max(window) and image[y, x] >= thresh * np.max(image):
15.                corner_list.append((x, y))
16.    return np.array(corner_list)
```

2.detail explanation:

- R depends only on eigenvalues of M
- R is large for a corner
- R is negative with large magnitude for an edge
- |R| is small for a flat region
- d > 0 is some threshold

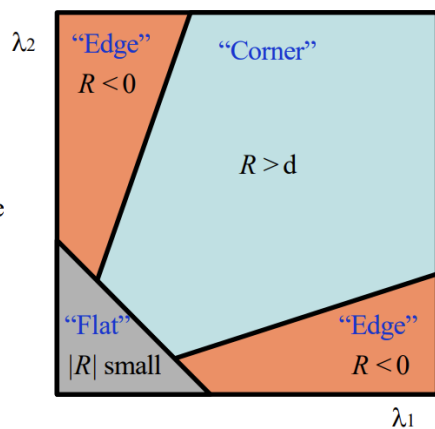


Fig1.3 from week4 slide

From the above Fig1.3, we can know that a large R value represents a corner point. The following will use non_max_suppression to get a larger R in a specific region.

- 1) Get the height and width of the input image.

```
h, w = image.shape
```

- 2) Computes the half of the window size, used to position the window at the pixel center.

```
offset = window_size // 2
```

- 3) Initialize an empty list corner_list to store the detected corner coordinates.

```
corner_list = []
```

- 4) The double loop traverses the pixels of the entire image, and each loop determines the coordinates (x, y) of the current pixel.

```
for y in range(offset, h - offset):  
    for x in range(offset, w - offset):
```

- 5) Get the local window centered on the current pixel.

```
        window = image[y-offset:y+offset+1, x -  
offset:x+offset+1]
```

- 6) Determine whether the value of the current pixel is equal to the maximum value of the local window, and whether it is greater than thresh times the global maximum value. If the conditions are met, add the current pixel coordinates (x, y) to the corner_list list.

```
if image[y, x] == np.max(window) and image[y, x] >= thresh*np.m  
ax(image):  
    corner_list.append((x, y))
```

- 7) After the loop ends, return the detected corner coordinate list corner_list.

4. Test this function on the provided four test images (Harris-[1,2,3,4].jpg, they can be downloaded from Wattle). Display your results by marking the detected corners on the input images (using circles or crosses, etc) (0.5 mark for each image, 2 marks in total). Note: Please make sure that your code can be run successfully on a local machine and generate results. If your submitted code cannot replicate your results, you will receive a 0 mark.

Implement method and code

1. Overview

```
#my result  
fig = plt.figure()  
ax = fig.add_subplot(121)  
bw_1 = cv2.imread('Task1/Harris-1.jpg', cv2.IMREAD_GRAYSCALE)  
R = Harris_cornerness(bw_1, sigma, k, dx, dy)  
corner_points = non_max_suppression(R, block_size, thresh)  
ax.set_title("Image1 of mine")  
ax.imshow(bw_1, cmap='gray')  
ax.plot(corner_points[:, 0], corner_points[:, 1], 'r.', markersiz  
e=3)  
ax2 = fig.add_subplot(122)
```

- 1) Read a grayscale image named "Task1/Harris-1.jpg" from the file and store it in the

variable "bw_1". The grayscale image is read because in the corner detection algorithm, only the grayscale value of the image needs to be considered, and the color information does not need to be considered. Therefore, converting images to grayscale images can reduce computational complexity and improve algorithm efficiency. Grayscale images can also save storage space.

```
bw_1 = cv2.imread('Task1/Harris-1.jpg', cv2.IMREAD_GRAYSCALE)
```

- 2) Calculate the corner response function value: Call the "Harris_cornerness" function to calculate the corner response function value and store it in the variable "R".

```
R = Harris_cornerness(bw_1,sigma, k, dx, dy)
```

- 3) Non-maximum suppression: Call the "non_max_suppression" function to get the coordinates of the corner points and store them in the variable "corner_points".

```
corner_points = non_max_suppression(R,block_size,thresh)
```

- 4) Draw the image with corner detection results

```
ax.plot(corner_points[:, 0], corner_points[:, 1], 'r.', markersize=3)
```

Below is the corner detection result of my function.

I drew the corner_point on the original image of the image, which is the red point. This represents the result of image corner detection.

Code explanation

- (1) create a new figure window.

```
fig = plt.figure()
```

- (2) Add a subplot to the figure with 1 row, 2 columns, and index 1.

```
ax = fig.add_subplot(121)
```

(3) read image as a grayscale image. Since corner detection only needs grayscale values when calculating gradients, reading grayscale images is beneficial to improve computational efficiency and reduce computational complexity.

```
bw_1 = cv2.imread('Task1/Harris -  
1.jpg', cv2.IMREAD_GRAYSCALE)
```

- (4) Call the corner detection function to return the corner detection matrix.

```
R = Harris_cornerness(bw_1,sigma, k, dx, dy)
```

- (5) Call the Non_max_suppression function to find the corner.

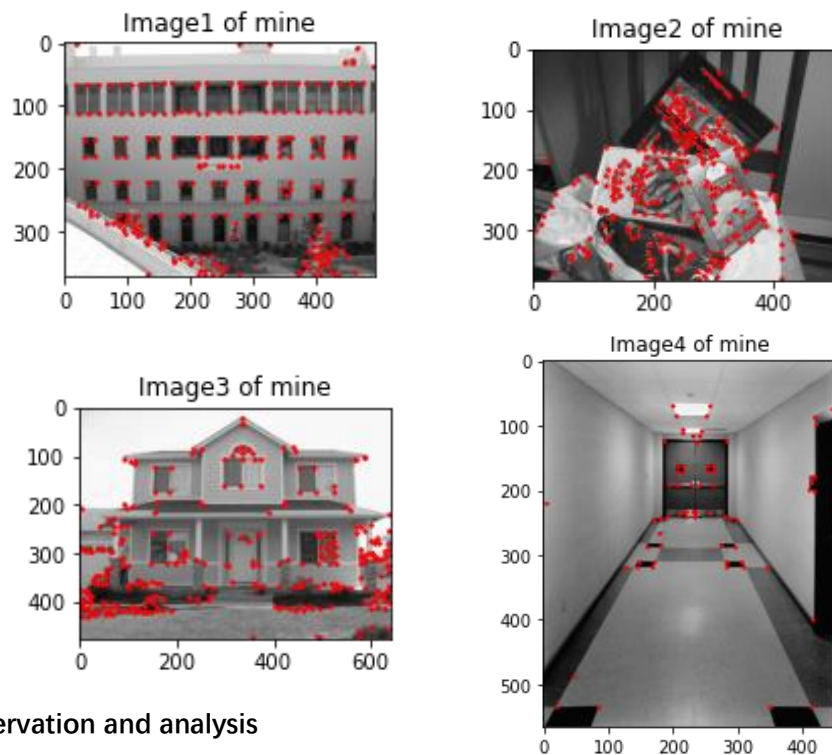
```
corner_points = non_max_suppression(R,block_size,thresh)
```

- (6) show the corner_point as red points in image.

```
ax.plot(corner_points[:, 0], corner_points[:, 1], 'r.', mar  
kersize=3)
```

Results visualization

The test results are as shown in the figure below.



Observation and analysis

Image1:

Image1 shows the window corners of image1, the corners of the building, and the tips of the branches of the tree are also marked with red dots, indicating that my corner detection function successfully detected most of the corner points.

Image2:

The corners of the book in Figure 2, the back and the corners, and the corners on the book cover are marked with red dots.

Image3:

The building corners of the villa, as well as the corners on the windows and some arcs are marked as red dots.

Image4:

The door frame, the square pattern on the door, and the four corners of the black tiles on the ground, are all marked with red dots.

Combining the observations of each of the above pictures, it can be concluded that the more obvious corner points can be marked, and the function has a good performance on different pictures.

5. Compare your results with that from Python's built-in function `cv2.cornerHarris()` (0.5 mark). Note: You should show your corner detection results and their comparisons for each of the test images.

Implement method and code

(1) I used my own `Harris_cornerness` function and the Python's built-in function `cv2.cornerHarris()` to calculate respectively. The steps to use my function have been explained in the previous question, so only the steps to use the built-in function are shown.

```
dst = cv2.cornerHarris(bw_1, block_size, ksize, k)
```

(2) Then put the two obtained R matrices into the Non-max function, got the corner points.

```
corner_points = non_max_suppression(dst, block_size, thresh)
```

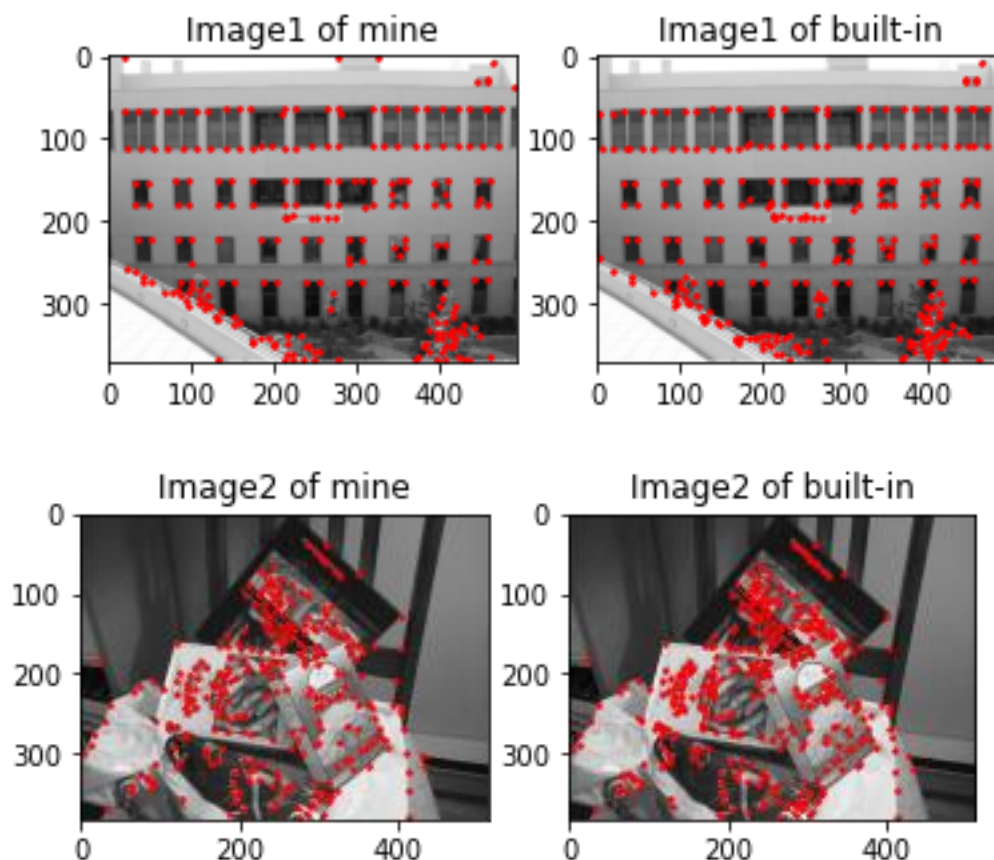
(3) marked them with red dots on the two graphs.

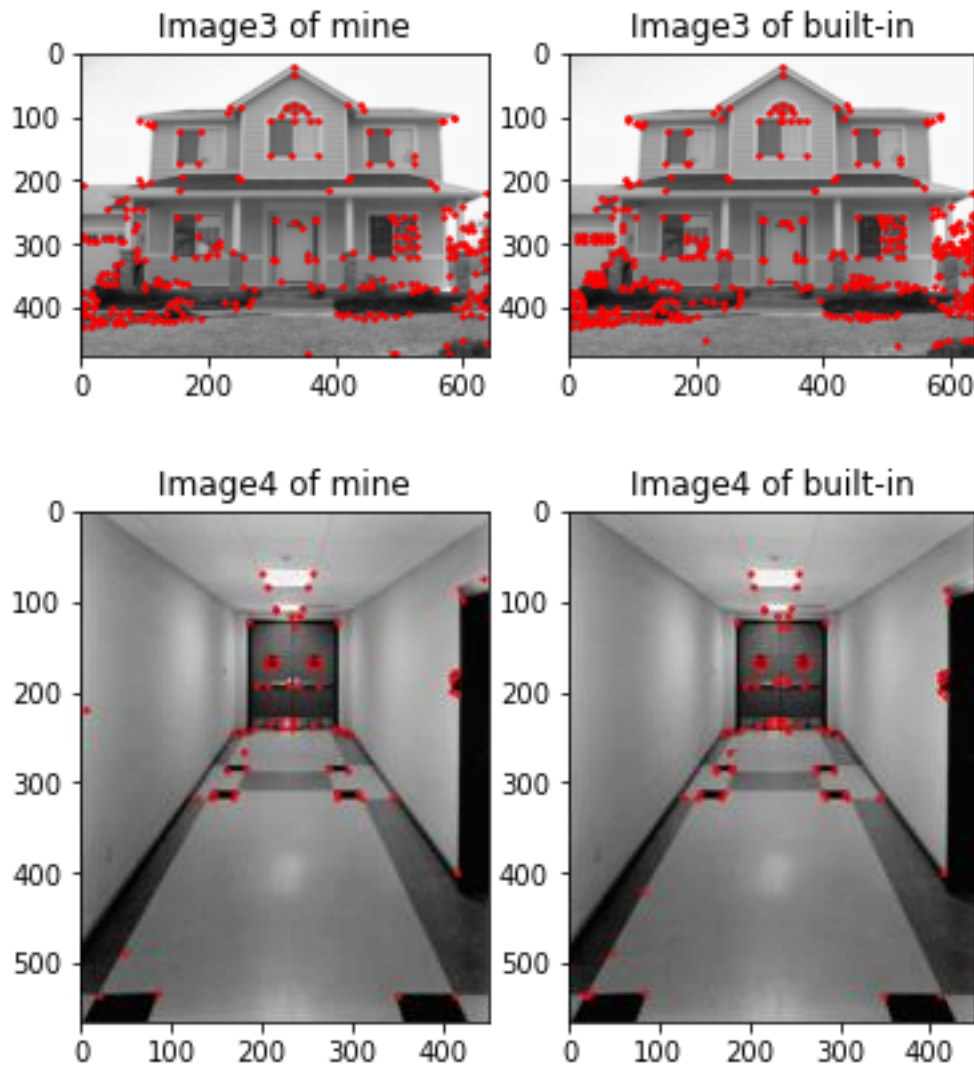
```
ax.plot(corner_points[:, 0], corner_points[:, 1], 'r.', markersize=3)
```

```
ax2.plot(corner_points[:, 0], corner_points[:, 1], 'r.', markersize=3)
```

Results visualization

The results are as follows, the picture on the left represents my corner detection result, and the right represents the corner detection result of the built-in function.





Observation and analysis

Image 1 comparison

The commonality between my function and the built-in function is that we have marked the corners of windows, building branches, and highways, but the difference is that at the location of the chimney on the roof, I marked the corners of the chimney but the built-in Function does not. In addition, the built-in function will detect more points in places where the focus is denser, such as tree branches.

Image 2 comparison

The same point is almost the same as image1 comparison, so I won't elaborate too much. It can be seen that the result of my corner detection function is not as dense as the built-in function. On the pattern of the book, the built-in function can detect more detailed corner points.

Image 3 comparison

Same as above. The difference is that the built-in function can detect more corners in the position of the bush.

Image 4 comparison

The detection results of these two pictures are almost the same. The obvious difference

is that the position of the edge of the picture is the corner point of the border in my function.

Perhaps due to the difference in parameters and the built-in algorithm, it can be seen that most of the corner detection results I get are the same as those of the built-in function, and there are some discrepancies in a small number of points.

6. **Implement the inverse image warping function which takes an input image, (inverted) transformation matrix and the output image size as inputs. Then, it returns the transformed image (0.5 mark). Note: For any inverse projection not lying on the original input image pixels, you should be using bilinear interpolation to calculate the pixel value. You should not use any library functions (e.g., `cv2.warpAffine()`) to directly perform inverse warping or bilinear interpolation, or you will get a 0 mark.**

Implement method and code

The function of this function is to reverse the perspective transformation of the input image according to the input inverse transformation matrix, and output a new image of the specified size. The specific implementation process is to traverse each pixel of the new image, and correspond it to the original image through the inverse transformation matrix, and then calculate the pixel value of the corresponding pixel in the original image through bilinear interpolation, and set it to the new image The value of the current pixel. Finally return the new image after inverse transformation.

Overview

```
1. def inverse_warp(input_image, inv_transform, output_size):
2.     # Get the width and height of the input image
3.     input_width, input_height = input_image.size
4.     # create a object as output
5.     transformed_img = Image.new(input_image.mode, output_size)
6.     #calculate the new pixel after transform
7.     inv_transform = np.linalg.inv(inv_transform)
8.     for y_transformed in range(output_size[1]):
9.         for x_transformed in range(output_size[0]):
10.            #(x_transformed, y_transformed) converted to homogeneous coordinate form (x_transformed, y_transformed, 1)
11.            transformed_point = np.array([x_transformed, y_transformed, 1])
12.            #Calculate the homogeneous coordinates (x_input, y_input, w_input) in the corresponding pre-transformed image
13.            #by solving the product of the matrix inv_transform and the homogeneous coordinates (x_transformed, y_transformed, 1)
14.            input_point = np.dot(inv_transform, transformed_point)
```

```

15.         #w_input represents the scaling factor in homogenous coordinates. Finally, divide x_input and y_input by w_input
16.         x_input, y_input = input_point[:2] / input_point[2]
17.         # If the coordinates in the transformed image are within the range of the transformed image, perform bilinear interpolation to calculate the pixel value
18.         if x_input >= 0 and x_input < input_width - 1 and y_input >= 0 and y_input < input_height - 1:
19.             x1 = int(x_input)
20.             #Calculate the corresponding coordinates in the pre-transformed image based on the transformed coordinates
21.             y1 = int(y_input)
22.             x2 = x1 + 1
23.             y2 = y1 + 1
24.             dx = x_input - x1
25.             dy = y_input - y1
26.             #Get the pixel values of four adjacent pixels
27.             pixel_value_00 = input_image.getpixel((x1, y1))
28.             pixel_value_01 = input_image.getpixel((x1, y2))
29.             pixel_value_10 = input_image.getpixel((x2, y1))
30.             pixel_value_11 = input_image.getpixel((x2, y2))
31.             #Calculate the pixel value by bilinear interpolation based on the values of the four pixels and the offset of the transformed coordinates
32.             pixel_value = (
33.                 (1 - dx) * (1 - dy) * pixel_value_00[0] + dx * (1 - dy) * pixel_value_10[0] + (1 - dx) * dy * pixel_value_01[0] + dx * dy * pixel_value_11[0],
34.                 (1 - dx) * (1 - dy) * pixel_value_00[1] + dx * (1 - dy) * pixel_value_10[1] + (1 - dx) * dy * pixel_value_01[1] + dx * dy * pixel_value_11[1],
35.                 (1 - dx) * (1 - dy) * pixel_value_00[2] + dx * (1 - dy) * pixel_value_10[2] + (1 - dx) * dy * pixel_value_01[2] + dx * dy * pixel_value_11[2]

```

```

        dy) * pixel_value_10[2] + (1 -
        dx) * dy * pixel_value_01[2] + dx * dy * pixel_value_11[2]
36.            )
37.            # Set the calculated pixel values into the
        new image
38.            transformed_img.putpixel((x_transformed, y_
        transformed), (int(pixel_value[0]), int(pixel_value[1]), in
        t(pixel_value[2])))
39.    return transformed_img

```

The guideline of my function

- 1) Define the function `inverse_warp` that takes three arguments:
- 2) `input_image`: the input image to be transformed
- 3) `inv_transform`: the inverse transformation matrix to be applied to the input image
- 4) `output_size`: the size of the output image
- 5) Get the width and height of the input image using the `size` attribute.


```
input_width, input_height = input_image.size
```
- 6) Create a new image object to hold the transformed image using the `Image.new` function.


```
transformed_img = Image.new(input_image.mode, output_size)
```
- 7) Iterate over every pixel in the output image by looping through the `x_transformed` and `y_transformed` coordinates using the `range` function.


```
for y_transformed in range(output_size[1]):
    for x_transformed in range(output_size[0]):
```
- 8) Convert the `x_transformed` and `y_transformed` coordinates to homogeneous coordinate form (with a third dimension of 1) using the `np.array` function.


```
transformed_point = np.array([x_transformed, y_tran
sformed, 1])
```
- 9) Calculate the corresponding pre-transformed image coordinates (`x_input`, `y_input`) by multiplying the inverse transformation matrix `inv_transform` with the homogeneous coordinates (`x_transformed`, `y_transformed`, 1) using the `np.dot` function.


```
input_point = np.dot(inv_transform, transformed_point)
```
- 10) Convert the pre-transformed image coordinates to Cartesian form by dividing the first two elements of the resulting vector by the third element.


```
x_input, y_input = input_point[:2] / input_point[2]
```
- 11) Check if the `x_input` and `y_input` values are within the range of the pre-transformed image using `if` statements.


```
if x_input >= 0 and x_input < input_width -
1 and y_input >= 0 and y_input < input_height - 1:
```
- 12) If the `x_input` and `y_input` values are within range, perform bilinear interpolation to calculate the pixel value:
- 13) Calculate the indices of the four adjacent pixels using `int` function and the corresponding `x_input` and `y_input` values.

```

y1 = int(y_input)
x2 = x1 + 1
y2 = y1 + 1

```

14) Calculate the fractional offset of the transformed coordinates using the difference between the `x_input` and `x1` and `y_input` and `y1`.

```

dx = x_input - x1
dy = y_input - y1

```

15) Get the pixel values of the four adjacent pixels using the `getpixel` method of the `input_image` object.

```

pixel_value_00 = input_image.getpixel((x1, y
1))
pixel_value_01 = input_image.getpixel((x1,
y2))
pixel_value_10 = input_image.getpixel((x2,
y1))
pixel_value_11 = input_image.getpixel((x2,
y2))

```

16) Calculate the pixel value using bilinear interpolation based on the values of the four pixels and the offset of the transformed coordinates.

```

pixel_value = (
    (1 - dx) * (1 -
dy) * pixel_value_00[0] + dx * (1 -
dy) * pixel_value_10[0] + (1 -
dx) * dy * pixel_value_01[0] + dx * dy * pixel_value_11[0],
    (1 - dx) * (1 -
dy) * pixel_value_00[1] + dx * (1 -
dy) * pixel_value_10[1] + (1 -
dx) * dy * pixel_value_01[1] + dx * dy * pixel_value_11[1],
    (1 - dx) * (1 -
dy) * pixel_value_00[2] + dx * (1 -
dy) * pixel_value_10[2] + (1 -
dx) * dy * pixel_value_01[2] + dx * dy * pixel_value_11[2]
)

```

17) Set the calculated pixel values into the new image using the `putpixel` method.

```

transformed_img.putpixel((x_transformed, y_
transformed), (int(pixel_value[0]), int(pixel_value[1]), in
t(pixel_value[2])))

```

7. Please select one of the four images (Harris-[1,2,3,4].jpg), and rotate it by 0, 90, 180, and 270 degrees clockwise using your image warping function above. Then, apply your Harris corner detection algorithm to the resulting images. Record the coordinates of the detected corners, compare them across the rotations, and report your observations and explanations in your report (0.5 mark)

Rotations

Implement method and code

I selected image 3 for rotating. Using the function in T1 q6.

To get the Harris-3 rotated by 0, 90, 180, 270 degrees clockwise. The transform matrix is as follow:

Rotate 0 degree

```
if angle == 0:
    transform = np.array([[1, 0, 0],
                          [0, 1, 0],
                          [0, 0, 1]])
```

Rotate 90 degree

```
elif angle == 90:
    transform = np.array([[0, 1, 0],
                          [-1, 0, input_image.size[1]-1],
                          [0, 0, 1]])
```

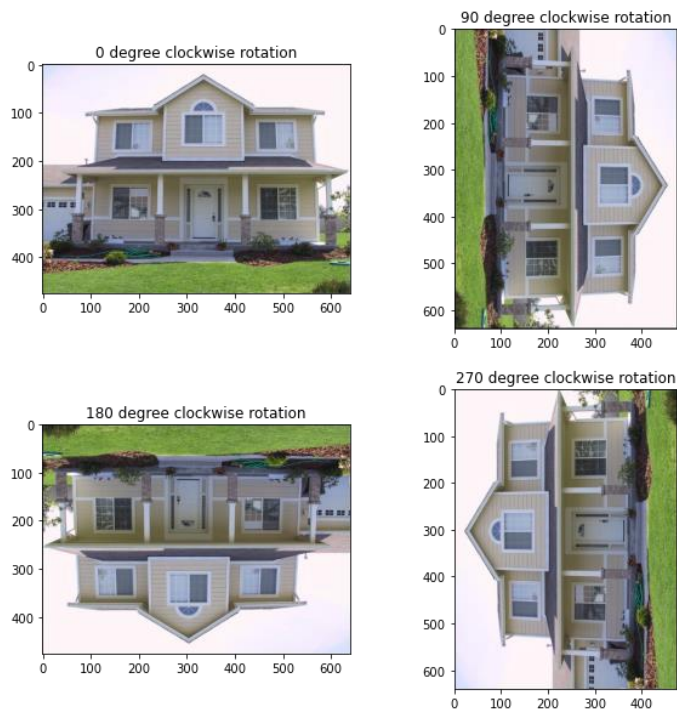
Rotate 180 degree

```
elif angle == 180:
    transform = np.array([[-1, 0, input_image.size[0]-1],
                          [0, -1, input_image.size[1]-1],
                          [0, 0, 1]])
```

Rotate 270 degree

```
elif angle == 270:
    transform = np.array([[0, -1, input_image.size[0]-1],
                          [1, 0, 0],
                          [0, 0, 1]])
```

Rotation result visualization



Harris corner detection for 4 rotated image

Implement method and code

The same method as above for corner detection of pictures, I put four pictures into the `Harris_cornerness(detail in Task1 (3))` function and use the `non_max_suppression` function to select the corner points, it is best to mark the corner points on the picture in the form of red dots.

Result visualization

The image result obtained by rotating image 3 by 0, 90, 180, 270 degrees

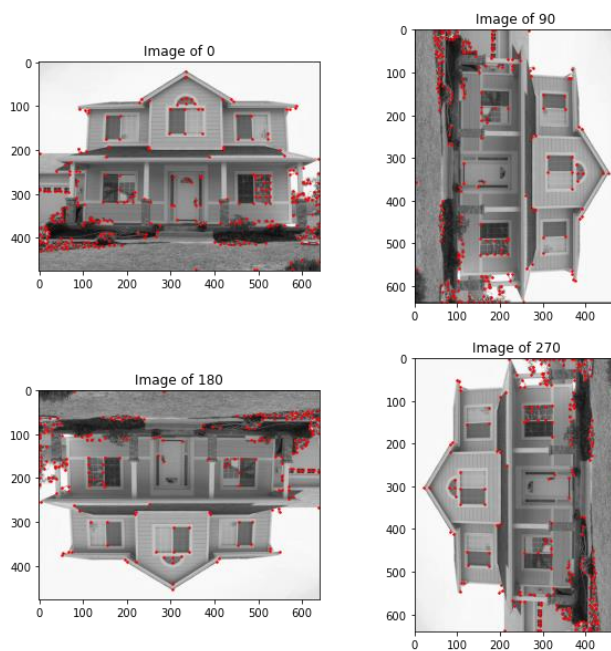


Fig Image three rotations 0, 90, 180, 270 degree angle detection results

Observation and analysis

We rotated the original images by 90° , 180° and 270° and applied the Harris corner detection algorithm to each image. By recording the detected corner coordinates, observations show that the Harris corner detection algorithm successfully detects salient corners in both the original image and its rotated version. However, in the rotated image, the coordinates of these corners are changed. This is because the image rotation causes the spatial position of objects and textures in the image to change, which affects the corner detection results. Nevertheless, under different rotation angles, the number of detected corners and their relative positions remain consistent, which indicates that the Harris corner detection algorithm has certain invariance to rotation.

In summary, the experimental results show that the Harris corner detection algorithm can effectively detect salient corners in both the original image and its rotated version. Although the corner coordinates change with image rotation, the algorithm still exhibits robustness to rotation. In practice, this means that the Harris corner detection algorithm can provide valuable information for image feature extraction under different viewing angles and orientations.

Task-2 Domain Adaptation.

1. Complete the following steps for preparation.

a. Download the dataset directly from Wattle or execute one of the data downloading scripts (dataset_downloader.sh or dataset_downloader.py).

I selected download the file from wattle. The data file in my assignment folder.

b. Implement a PyTorch Dataset class for the dataset and load the data with appropriate transformations (0.5 mark).

First, I defined a PyTorch dataset class for the image data, named CustomDataset. CustomDataset dataset inherits the torch.utils.data.Dataset class, which can easily customize the dataset and realize free control over the data. Then I use the custom dataset class CustomDataset to load data, and create 4 DataLoader objects for iterating training and testing datasets.

Implement method and code

b.1 Overview

The dataset(CustomDataset) also defines three basic methods.

1. init
2. len
3. getitem

```
1. class CustomDataset(torch.utils.data.Dataset):
2.     def __init__(self, root_dir, transform=None):
3.         self.root_dir = root_dir
4.         self.transform = transform
5.         self.images = [] # A list to store the paths of all
                           # images in the dataset
6.         self.labels = [] # A list to store the labels of all
                           # images in the dataset
7.         # Only read .jpg format image files
8.         self.image_filenames = [f for f in os.listdir(root_dir) if f.lower().endswith('.jpg') and not f.lower().startswith('.')]
9.         # Enumerate the files in each folder and label the
                           # images for each category
10.        for i, class_name in enumerate(os.listdir(root_dir)):
11.            class_dir = os.path.join(root_dir, class_name)
12.            if os.path.isdir(class_dir):
13.                for img_name in os.listdir(class_dir):
```

```

14.             if img_name.lower().endswith('.jpg'):
15.                 img_path = os.path.join(class_dir,
                    img_name)
16.                 self.images.append(img_path)
17.                 self.labels.append(i)
18.     def __len__(self):
19.         return len(self.images)
20.     def __getitem__(self, idx):
21.         img_path = self.images[idx] # Get the path of the i
            mage
22.         label = self.labels[idx] # Get the Label of the ima
            ge
23.         img = Image.open(img_path).convert("RGB") # Open th
            e image and convert it to RGB format
24.         # If a transform object is passed in, use it to pre
            process the image
25.         if self.transform is not None:
26.             img = self.transform(img)
27.         return img, label # Return the preprocessed image a
            nd its Label

```

b.2 Detail explanation

init()

Initialize function, pass in the path of the dataset and the transform object for data preprocessing.

- (1) The method is used to initialize the dataset object. It accepts two parameters root_dir and transform, where root_dir is the root directory containing image files and transform is an optional parameter for image transformation.

```

        self.root_dir = root_dir
        self.transform = transform

```

- (2) Initialize two lists, used to store the images from path dataset and the labels of images.

```

        self.images = [] # A list to store the paths of all images i
            n the dataset
        self.labels = [] # A list to store the labels of all images
            in the dataset

```

- (3) If this line of code is not added, the hidden file will be read in, so that all image files are read in

```

        self.image_filenames = [f for f in os.listdir(root_dir) if f
            .lower().endswith('.jpg') and not f.lower().startswith('.')]

```

- (4) Traverse all the folders under the dataset path, and use the enumerate() function to convert the folder name and number into a tuple (i, class_name)

```

        for i, class_name in enumerate(os.listdir(root_dir)):

```

- (5) Traverse all the files (pictures and other files) in the folder, keep only the pictures ending with '.jpg' in the file name, add the path of the picture to the list of self.images, and

label i corresponding to the picture (Generated in the enumerate() function) added to the self.labels list.

```
for i, class_name in enumerate(os.listdir(root_dir)):
    class_dir = os.path.join(root_dir, class_name)
    if os.path.isdir(class_dir):
        for img_name in os.listdir(class_dir):
            if img_name.lower().endswith('.jpg'):
                img_path = os.path.join(class_dir, img_name)
                self.images.append(img_path)
                self.labels.append(i)
```

len()

It is used to return the number of images in the dataset.

getitem()

This method is used to get the image at the specified index idx and its corresponding label.

(1) Get the path and label of image

```
img_path = self.images[idx] # Get the path of the image
label = self.labels[idx] # Get the label of the image
```

(2) convert image as RGB form

```
img = Image.open(img_path).convert("RGB") # Open the image and
convert it to RGB format
```

(3) use transform function process the image.

```
if self.transform is not None:
    img = self.transform(img)
```

transform

Mainly the following three basic operations

1. Scale image to 224x224 pixel size
2. convert image to PyTorch Tensor type
3. normalize image, it can scale the pixel value to the range of [-1, 1], and make the mean value of the pixel value of the three channels be [0.485, 0.456, 0.406], and the variance is [0.229, 0.224, 0.225]

```
# Define the transformations
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])
```

Load data

Get data and transform the data into dataset, then create 4 Dataloader object for preparing.

(1) Four datasets were loaded using CustomDataset, including real_train_dataset, real_test_dataset, sketch_train_dataset, and sketch_test_dataset.

(3984 images in the real_train folder, 1712 in the real_test folder, 1956 in the sketch_train folder and 841 in the sketch_test folder)

```

real_train_dataset = CustomDataset('data/real_train', transform=transform)
real_test_dataset = CustomDataset('data/real_test', transform=transform)
sketch_train_dataset = CustomDataset('data/sketch_train', transform=transform)
sketch_test_dataset = CustomDataset('data/sketch_test', transform=transform)

```

(2) Use DataLoader to load the training set and test set into memory, and create 4 DataLoader objects, namely real_train_loader, real_test_loader, sketch_train_loader, and sketch_test_loader, which are used to load data in batches and iterate while training the neural network. Among them, batch_size=32 means that the amount of data loaded in each batch is 32 samples. shuffle=True means whether to randomly shuffle the data before the start of each epoch to realize random sampling and enhancement of the data. Shuffle the train set is enough.

```

real_train_loader = DataLoader(real_train_dataset, batch_size=32, shuffle=True)
real_test_loader = DataLoader(real_test_dataset, batch_size=32, shuffle=False)
sketch_train_loader = DataLoader(sketch_train_dataset, batch_size=32, shuffle=True)
sketch_test_loader = DataLoader(sketch_test_dataset, batch_size=32, shuffle=False)

```

c. Load the pretrained ResNet-34 model from torchvision and modify its final fully connected layer to accommodate the specific number of classes required for the current task (0.5 mark).

Implement method and code

c.1 Overview

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# total ten class
num_classes = 10
#load the pretrain resnet34 model
model = models.resnet34(pretrained=True)
#get the features num
num_fts = model.fc.in_features
#set new model parameters
model.fc = nn.Linear(num_fts, num_classes)
# Move model to device
model = model.to(device)
# Define optimizer and Loss function
optimizer = optim.Adam(model.parameters(), lr=1e-4)
criterion = nn.CrossEntropyLoss()

```

c.2 detail explanation

Define what needs to be trained for Resnet-34 model

- 1) Define calculation device. If CUDA is available, use GPU; otherwise use CPU.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

- 2) Set the number of model categories to 10, categories contain backpack, book, car, pizza, sandwich, snake, sock, tiger, tree, watermelon. And get the feature numbers in model.

```
num_classes = 10
num_ftrs = model.fc.in_features
```

- 3) Load ResNet34 as pretrained model.

```
model = models.resnet34(pretrained=True)
```

- 4) Then replace the last fully connected layer of the model with a new fully connected layer with an output size of 10 for classification.

```
model.fc = nn.Linear(num_ftrs, num_classes)
```

- 5) Move data into GPU for calculation

```
model = model.to(device)
```

- 6) Using Adam optimizer with learning rate 1e-4.

```
optimizer = optim.Adam(model.parameters(), lr=1e-4)
```

- 7) Model training using the cross-entropy loss function as the loss function.

```
criterion = nn.CrossEntropyLoss()
```

2. Implement the code for fine-tuning a model.

a. Implement the `train()` and `test()` functions for fine-tuning (1 mark). Hint: Two functions could have the following input parameters as an example (`src_loader`, `tar_loader`, `model`, `optimizer`). Note that it is not a restriction on your function interface, but some suggestions. You are free to include additional parameters as needed.

Implement method and code

a.1 Train()

This function optimizes the model based on the loss between predicted and true labels.

`src_loader`: a PyTorch DataLoader object that loads the source domain training dataset.

`tar_loader`: a PyTorch DataLoader object that loads the target domain training dataset.

(Note: this parameter is currently not used in the function.)

`model`: the PyTorch model that will be trained.

`optimizer`: the PyTorch optimizer used for updating the model parameters during training.

`criterion`: the loss function used to calculate the difference between predicted and true labels.

a.1.1 overview

```
1. def train(src_loader, tar_loader, model, optimizer, criterion):
2.     #tar_loader is useless
3.     # Set model as train state
4.     model.train()
5.     for src_batch, tar_batch in zip(src_loader, tar_loader):
6.         # Extract source and target images and labels
7.         src_images, src_labels = src_batch
8.         tar_images, _ = tar_batch
9.         # Move images and labels to the device (CPU or GPU)
10.        src_images, src_labels = src_images.to(device), src_labels.to(device)
11.        tar_images = tar_images.to(device)
12.        # Zero out gradients in the optimizer
13.        optimizer.zero_grad()
14.        # get the prediction result
15.        src_outputs = model(src_images)
16.        tar_outputs = model(tar_images)
17.        # Calculate loss on source data
18.        loss = criterion(src_outputs, src_labels)
19.        #update
20.        loss.backward()
21.        optimizer.step()
```

a.1.2 Detail explanation

1) Set the model as the train state using `model.train()`.

```
    model.train()
```

- 2) Extract the source and target images and labels from the current batch.

```
    for src_batch, tar_batch in zip(src_loader, tar_loader):
```

- 3) Move the images and labels to the device (CPU or GPU) using to(device).

```
        src_images, src_labels = src_images.to(device), src_labels.to(device)
        tar_images = tar_images.to(device)
```

- 4) Zero out the gradients in the optimizer using optimizer.zero_grad().

```
        optimizer.zero_grad()
```

- 5) Get the prediction result by passing the source images through the model using model(src_images).

```
        src_outputs = model(src_images)
        tar_outputs = model(tar_images)
```

- 6) Calculate the loss on source data using criterion(src_outputs, src_labels).

```
        loss = criterion(src_outputs, src_labels)
```

- 7) Backpropagate the loss and update the model parameters using loss.backward() and optimizer.step().

```
        loss.backward()
        optimizer.step()
```

a.2 Test()

This function is used to test the model performance. And return the accuracy model prediction.

a.2.1 Overview

```
1. def test(loader, model):
2.     # Set model as evaluation state
3.     model.eval()
4.     #init for calculate the accuracy
5.     correct = 0
6.     total = 0
7.     # Turn off gradient calculation because we're only testing the model
8.     with torch.no_grad():
9.         for images, labels in loader:
10.             images, labels = images.to(device), labels.to(device)
11.             #get prediction from test image
12.             outputs = model(images)
13.             #calculate loss and acc
14.             _, predicted = torch.max(outputs.data, 1)
15.             total += labels.size(0)
16.             correct += (predicted == labels).sum().item()
17.     accuracy = 100. * correct / total
18.     return accuracy
```

a.2.2 Detail Explanation

- 1) Set the model to evaluation state

- ```

 model.eval()
2) Initialize the counter of the sum of correct rate, total number of samples
 correct = 0
 total = 0
3) Turn off the gradient calculation, since we are only testing the model
 with torch.no_grad():
4) Traverse the test data loader
 for images, labels in loader:
5) Move images and labels to device (CPU or GPU)
 images, labels = images.to(device), labels.to(device)
6) Get predict result from model
 outputs = model(images)
7) Use the torch.max function to get the predicted values for each sample from the output
 tensor outputs.
 _, predicted = torch.max(outputs.data, 1)
8) Add the size of the label tensor labels to the total variable, which is used to finally
 calculate the average accuracy of the model.
 total += labels.size(0)
9) Count the number of correctly predicted samples into the variable correct
 correct += (predicted == labels).sum().item()
10) Get accuracy
 accuracy = 100. * correct / total

```

**b. Train the ResNet-34 model on the Sketch domain (use images in sketch\_train), assuming all training labels are given. Then, evaluate the model also on the Sketch domain (use images in sketch\_test) (1 mark).**

### Implement method and code

I will train with the sketch training set, then test with the sketch test set. Since the domains of the training set and the test set are the same, they are all sketch types. So the correct rate on the test set will also be high.

Using the Resnet34 model defined above, I use the train() function adjust the model ten test, and use test() function to observe the accuracy difference during ten epochs. As follow is my operation explanation.

#### b.1 Overview

```

for epoch in range(10):
 #train model
 train(sketch_train_loader, sketch_train_loader, model, optimizer, criterion)
 #get a accuracy of new model
 sketch_test_accuracy = test(sketch_test_loader, model, criterion)
 #add accuracy to list, used for plot a accuracy graph
 sketch_test_accs.append(sketch_test_accuracy/100)

```

```
print(f'Epoch {epoch+1} -
Sketch Test Accuracy on ketch train: {sketch_test_accuracy:
.2f}%.'
```

## b.2 Detail explanation

(1) A loop of ten optimizations on the model

```
for epoch in range(10):
```

(2) train the model using the function train() above

```
train(sketch_train_loader, sketch_train_loader, model, optimizer, crite
rion)
```

(3) get the accuracy of new model

```
sketch_test_accuracy = test(sketch_test_loader, model, criterion)
```

(4) store every accuracy of ten epochs, used for draw graph.

```
sketch_test_accs.append(sketch_test_accuracy/100)
```

## b.3 results visualization

The table2(b) 1 shows the value of the accuracy of 10 trainings on the sketch test, and the Fig 2(b) 1 is a line graph of ten epochs.

| Epoch    | 1      | 2      | 3      | 4      | 5      | 6      | 7      | 8      | 9      | 10     |
|----------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|
| accuracy | 86.68% | 87.99% | 88.47% | 86.44% | 86.92% | 88.82% | 88.11% | 88.11% | 86.80% | 87.63% |

table 2(b) 1

The accuracy when the training set is sketch on the sketch testing set during ten iterations

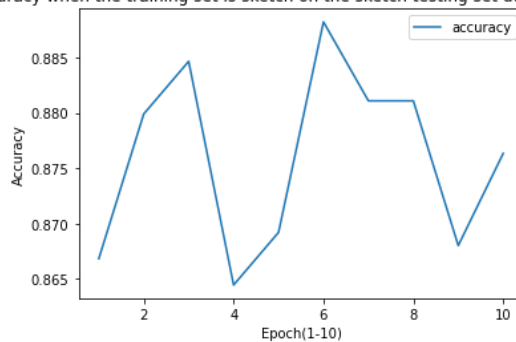


Fig 2(b) 1

## b.4 observation and analysis

Will be described in detail in d.

c. Train the ResNet-34 model on the Real domain (use images in real\_train), assuming all training labels are given. Then, evaluate the model also on the Sketch domain (use images in sketch\_test) (1 mark).

c.1 Explanation

I will create new model which haven't been trained. And use this new model train on the real training set and test it on sketch testing set. Since the training set is real and the test set is sketch, the domain between the two is different, so the prediction results obtained by the test set may be poor. The steps are similar to 2.2(b). The only difference is the dataset.

```
real_train_loss, real_train_accuracy = train(real_train_loader, real_train_loader, model, optimizer, criterion)
```

c.2 Results visualization

The figure and numbers below are the accuracy the model trained on the real training set, which has undergone ten iterations of optimization.

| Epoch    | 1      | 2      | 3      | 4      | 5      | 6      | 7      | 8      | 9      | 10     |
|----------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|
| accuracy | 47.32% | 46.37% | 46.61% | 42.57% | 50.77% | 46.49% | 42.93% | 46.37% | 49.70% | 48.63% |

table 2(c) 1

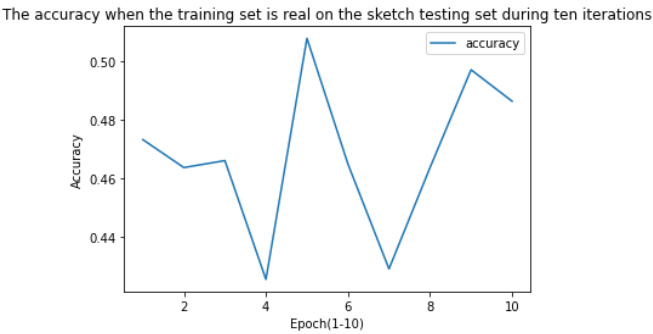


fig 2(c) 1

c.3 Observation and analysis

Will be described in detail in d.

d. Document the accuracy values obtained from steps 2(b) and 2(c) on the test set (i.e., sketch\_test) in your report. Compare and contrast these results and provide an explanation for any observation (1 mark).

#### d.1 Result visualization of (b)

The correct log file is stored in the same directory as log\_file\_2b.txt.

The table shows the value of the accuracy of 10 trainings on the sketch test, and the picture is a line graph of ten epochs.

| Epoch    | 1      | 2      | 3      | 4      | 5      | 6      | 7      | 8      | 9      | 10     |
|----------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|
| accuracy | 86.68% | 87.99% | 88.47% | 86.44% | 86.92% | 88.82% | 88.11% | 88.11% | 86.80% | 87.63% |

table 2(b) 2

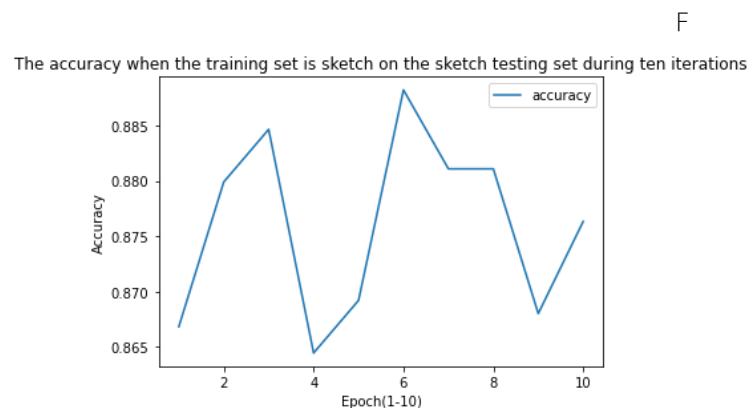


Fig 2(b) 2

#### d.2 observation and analysis of (b)

Calculate the average of 10 correct rates as follow,

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = 87.34\%$$

And the variance,

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 = 0.43\%$$

Among them,  $n$  represents the number of samples,  $x_i$  represents the  $i$ th sample,  $\bar{x}$  represents the sample mean, and  $s^2$  represents the sample variance.

The average value is 87.34% and the variance is 0.43%, which means that the accuracy the ten optimization models does not change much and is stable at a relatively high value.

Observing these data, it can be found that the accuracy the model on the test set fluctuates, but the overall performance is fairly stable. The highest accuracy rate appeared in the 6th model optimization, which was 88.82%, and the lowest accuracy rate appeared in the 4th and 9th model optimization, which were 86.44% and 86.80% respectively. During the whole process, the fluctuation range of the correct rate is not very large, and the correct rate is between 86% and 89% most of the time.

Analyzing this data, it may be found that the model performs best after the 6th model optimization, but worst after the 4th and 9th model optimizations. This may be because during the 6th model optimization, the model parameters were updated to a better state,

but during the 4th and 9th model optimization, some negative effects may have occurred, resulting in a decline in model performance.

### d.3 Result visualization of (c)

The correct log file is stored in the same directory as log\_file\_2c.txt.

The table shows the value of the accuracy of 10 trainings on the sketch test, and the picture is a line graph of ten epochs.

| Epoch    | 1      | 2      | 3      | 4      | 5      | 6      | 7      | 8      | 9      | 10     |
|----------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|
| accuracy | 47.32% | 46.37% | 46.61% | 42.57% | 50.77% | 46.49% | 42.93% | 46.37% | 49.70% | 48.63% |

table 2(c) 2

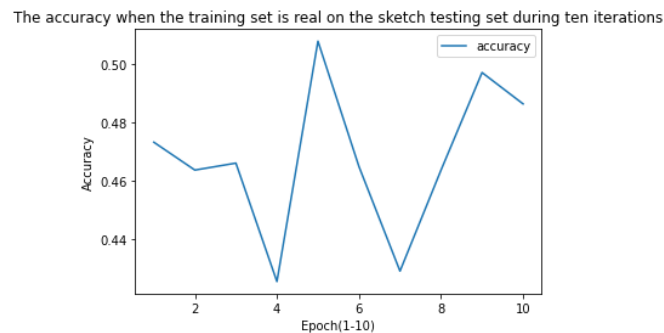


fig 2(c) 2

### d.4 Observation and analysis of (c)

Calculate the average of 10 correct rates as follow,

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = 46.78\%$$

And the variance,

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 = 6.12\%$$

Among them,  $n$  represents the number of samples,  $x_i$  represents the  $i$ th sample,  $\bar{x}$  represents the sample mean, and  $s^2$  represents the sample variance.

Based on the observations, it can be seen that the accuracy the model on the test set fluctuates widely, with a range between 42% and 50%. The highest accuracy rate appeared in the 5th model optimization, which was 50.77%, and the lowest accuracy rate appeared in the 4th model optimization, which was 42.57%. The rest of the accuracy rates are also relatively low, mostly between 46% and 49%. On the whole, the performance of the model on this task is not very good, the accuracy rate is low, the variance is relatively large, and the fluctuation range is relatively large.

It is worth noting that the correct rate in this set of data is generally low, and there is a large fluctuation range. The model has poor performance on this task. Compared with the first set of data, the performance of the model is less stable, and the accuracy rate is lower.

### d.5 Compare and contrast these results

It can be seen from observation that whether the model is trained on the sketch set or the real set, they have the same point that after ten model adjustments and optimizations, although the accuracy rate fluctuates up and down, the overall accuracy rate after training is still increase.

For example, the accuracy the model trained on the sketch dataset is 86.68% in the first test, and the accuracy the optimized model after ten iterations is 87.63%. The model trained on the real data set has a accuracy 47.32% for the first time and 48.63% for the tenth time.

The difference between them is that the correct rate obtained on the sketch training set is higher than that obtained on the real training set. The reason is that since the test set is a sketch, the model trained on the sketch has the same domain as the test set, so better results (higher than 80%) can be obtained. The model obtained on the real training set has a very low accuracy rate (less than 50%) due to the different domains of the training set and the test set.

### 3. Implement the code for domain adaptation.

a. You have two options of domain adaptation loss functions as follows. You can choose one of the two options in your implementation. In your code, add line-by-line comments and include them in your report.

#### Implement method and code

##### a.1 Method

I selected i maximum mean discrepancy(MMD).

This method is used to compare the distributions of source and target domains and reduce the difference between them by maximizing the distance between them.

Fig a.1 1 and Fig a.1 2 is from slide which indicated the detail of MMD.

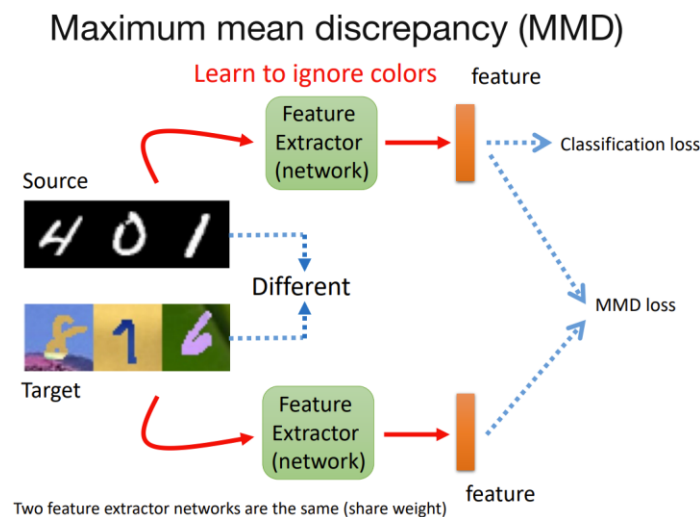


Fig a.1 1

#### Maximum mean discrepancy (MMD)

- In each training mini-batch, we compute the mean of source features:

$$\mu_S = \frac{1}{n_S} \sum_i f_S^i$$

$f_S^i$  is the feature of the  $i$ th source image  
 $n_S$  is the number of source features

- We compute the mean of target features

$$\mu_T = \frac{1}{n_T} \sum_i f_T^i$$

$f_T^i$  is the feature of the  $i$ th target image  
 $n_T$  is the number of target features

- The MMD loss is written as:

$$L_{MMD} = \|\mu_S - \mu_T\|^2.$$

Fig a.1 2

The following will explain the train function I rewrote to solve the domain adaption

#### a. 2 Code explanation

##### a.2.1 MMD loss

MMD is a measure of the difference between source domain data and target domain data, which is calculated by comparing the distance between the probability distributions in the two domains. In this code, MMD is used to measure the similarity between source domain data and target domain data for better model transfer. Specifically, when calculating MMD, it is necessary to calculate the distance between samples in each domain, that is, the kernels matrix, and use these matrices to calculate the value of MMD.

## Implement method and code

### a.2.1.1 mmd\_loss function

```
1. # move tensors to CPU device
2. x_src = x_src.detach().cpu().numpy()
3. x_tar = x_tar.detach().cpu().numpy()
4. # m and n is batch size of scr and tar 32
5. m = x_src.shape[0]
6. n = x_tar.shape[0]
7. # num of the feature
8. d = x_src.shape[1]
9. # calculate kernel matrices
10. K_ss = rbf_kernel(x_src, x_src)
11. K_tt = rbf_kernel(x_tar, x_tar)
12. K_st = rbf_kernel(x_src, x_tar)
13. # calculate MMD^2
14. mmd2 = (1 / (m * (m - 1))) * np.sum(K_ss -
 np.diag(np.diag(K_ss)))
15. mmd2 += (1 / (n * (n - 1))) * np.sum(K_tt -
 np.diag(np.diag(K_tt)))
16. mmd2 -= (2 / (m * n)) * np.sum(K_st)
17. return mmd2
```

### a.2.1.2 Detail Explanation

- (1) These lines of code move the PyTorch tensors x\_src and x\_tar from GPU device to CPU device and then convert them to NumPy arrays using the cpu() and numpy() methods respectively. This is necessary because the rbf\_kernel function in the mmd\_loss function expects NumPy arrays as input, not PyTorch tensors.

```
x_src = x_src.detach().cpu().numpy()
x_tar = x_tar.detach().cpu().numpy()
```

- (2) Get m and n for subsequent calculations, m represents the number of samples of source, n represents the number of samples of target. d is the dimensionality of features extracted from the image.

```
m = x_src.shape[0]
n = x_tar.shape[0]
num of the feature
```

```
d = x_src.shape[1]
```

- (3) These three kernel matrices are used to calculate the MMD (Maximum Mean Deviation) value, which is the core step in computing the MMD loss function.  $K_{ss}$  is the kernel matrix for source domain data,  $K_{tt}$  is the kernel matrix for target domain data, and  $K_{st}$  is the kernel matrix between the source and target domain data. For two samples  $x_i$  and  $x_j$ , the radial basis kernel function  $K(x_i, x_j)$  is calculated as follows, among them, we use the default value of gamma directly. In the code, we directly call `rbf_kernel`, which comes from `from sklearn.metrics.pairwise import rbf_kernel`

$$K(x_i, x_j) = \exp\left(-\gamma \|x_i - x_j\|^2\right)$$

```
calculate kernel matrices
K_ss = rbf_kernel(x_src, x_src)
K_tt = rbf_kernel(x_tar, x_tar)
K_st = rbf_kernel(x_src, x_tar)
```

- (4) Then we calculate MMD according to the following formula,

$$MMD^2 = \frac{1}{m(m-1)} \sum_{i \neq j} K(x_i, x_j) - \frac{2}{mn} \sum_{i,j} K(x_i, y_j) + \frac{1}{n(n-1)} \sum_{i \neq j} K(y_i, y_j)$$

```
14. mmd2 = (1 / (m * (m - 1))) * np.sum(K_ss -
 np.diag(np.diag(K_ss)))
15. mmd2 += (1 / (n * (n - 1))) * np.sum(K_tt -
 np.diag(np.diag(K_tt)))
16. mmd2 -= (2 / (m * n)) * np.sum(K_st)
17. return mmd2
```

### a.2.2 Get\_feature

This function is used to extract feature vectors from the input image. Pass the input image to the model, then extract the feature vectors in the convolutional layer and residual block layer of the model, concatenate them into a one-dimensional vector, and finally return this one-dimensional vector as the feature vector representation of the image.

#### a.2.2.1 Overview

```
1. def get_feature(model, inputs):
2. # Use convolutional layer 1 for feature extraction
3. features = model.conv1(inputs)
4. # Batch normalization
5. features = model.bn1(features)
6. # Apply ReLU activation function
7. features = model.relu(features)
8. # Max pooling layer
9. features = model.maxpool(features)
10. # Use residual block layer 1 for feature extraction
11. features = model.layer1(features)
12. # Use residual block layer 2 for feature extraction
```

```

13. features = model.layer2(features)
14. # Use residual block layer 3 for feature extraction
15. features = model.layer3(features)
16. # Use residual block layer 4 for feature extraction
17. features = model.layer4(features)
18. # Average pooling layer
19. features = model.avgpool(features)
20. # Flatten features into a 1D tensor
21. features = torch.flatten(features, 1)
22. # Return the feature vector
23. return features

```

#### a.2.2.2 Detail Explanation

The implementation principle of this function is relatively simple, and the code comments have given enough explanations, so I won't go into details.

#### a.2.3 train\_transfer

This function implements the training process of transfer learning between the source and target domains. Specifically, it receives data loaders for the source and target domains, models, optimizers, loss functions, and a parameter `lambda`, where `lambda` is a hyperparameter controlling how well the domains fit.

##### a.2.3.1 Overview

```

1. def train_transfer(src_loader, tar_loader, model, optimizer
 , criterion, lambda):
2. #set the model to train state
3. model.train()
4. # Iterate through batches of source and target data
5. for src_batch, tar_batch in zip(src_loader, tar_loader):
6. # Extract source images and labels, and target images
7. src_images, src_labels = src_batch
8. tar_images, _ = tar_batch
9. # Move data to GPU
10. src_images, src_labels = src_images.to(device), src_labels.to(device)
11. tar_images = tar_images.to(device)
12. # Zero the gradients for the optimizer
13. optimizer.zero_grad()
14. #get features from source image
15. src_features = get_feature(model,src_images)
16. #get features from target image
17. tar_features = get_feature(model,tar_images)
18. #get prediction from source
19. src_outputs = model(src_images)
20. # Calculate the loss function on the source domain
 data

```

```

21. src_loss = criterion(src_outputs, src_labels)
22. #Calculate mmd loss
23. m_loss = lambd * mmd_loss(src_features, tar_features)
24. # get total loss
25. loss = src_loss + mmd_loss
26. # Backpropagate the total loss to calculate gradients
27. loss.backward()
28. # Update model parameters using the optimizer
29. optimizer.step()

```

#### a.2.3.2 Detail Explanation

The step of transfer train is as follow.

- (1) Set the model into training state

```
model.train()
```

- (2) loop over batches of source and target data.

```
for src_batch, tar_batch in zip(src_loader, tar_loader):
```

- (3) Extract source images and labels, and target images.

```
src_images, src_labels = src_batch
tar_images, _ = tar_batch
```

- (4) Move data to the GPU. Moving the source images and labels to the GPU is to allow the model to calculate these data on the GPU and use the parallel computing power of the GPU to speed up the training.

```
src_images, src_labels = src_images.to(device), src_labels.to(device)
```

- (5) Clear the gradient for the optimizer.

```
optimizer.zero_grad()
```

- (6) Get feature matrix from source and target, this is a step for Feature Extractor(Network) .

```
#get features from source image
src_features = get_feature(model,src_images)
#get features from target image
tar_features = get_feature(model,tar_images)
```

- (7) Get predict from source image.

```
src_outputs = model(src_images)
```

- (8) Calculate the loss function on the source domain data

```
src_loss = criterion(src_outputs, src_labels)
```

- (9) Use the previously explained function mmd\_loss to calculate mmd,

```
m_loss = lambd * mmd_loss(src_features, tar_features)
```

- (10)Combine the loss of source data and mmd loss into a total loss. The product of lambd and the MMD loss value will be added to the cross-entropy loss value of the source domain data to calculate the total loss. The larger the value, the larger the proportion of MMD loss.

$$L = L_{\text{class}} + \lambda L_{\text{MMD}}$$

```
loss = src_loss + mmd_loss
```

(11) Pass the total loss backwards to compute gradients.

```
loss.backward()
```

(12) Use the optimizer to update model parameters..

```
optimizer.step()
```

### a. 3 Conclusion

The above code implements the training process of a deep learning-based domain adaptation model. The model uses a pre-trained convolutional neural network model and reduces the distribution difference between the source domain and the target domain through the Maximum Mean Discrepancy (MMD). During the training process, features are extracted for each batch of source domain data and target domain data, the loss function is calculated, and backpropagation and model parameter updates are performed.

**b. Based on the ResNet-34 backbone pretrained on ImageNet, perform unsupervised domain adaptation. Specifically, the Real domain (real\_train) is the source, and the Sketch domain (sketch\_train) is the target. You should use images and labels from the Real domain and images ONLY from the Sketch domain to train your model. You should apply stochastic gradient descent to minimize the joint loss function composed of cross-entropy and one of the domain adaptation loss functions you have chosen. After this domain adaptation training procedure, evaluate classifier performance (accuracy) on the Sketch domain (sketch\_test). Compare the new accuracy value with Task 2.2(c), describe any observations and provide your explanations (3 marks). Hint: To optimize the joint loss, it is necessary to specify the weight assigned to the domain adaptation loss function. For the mean discrepancy loss, it is recommended to set the weight to 0.01.**

### Implement method and code

#### b.1 Method

In fact, I did not use the stochastic gradient descent optimizer, but the Adam optimizer. (Adam optimizer is allowed be used, piazza link: <https://piazza.com/class/lDy9z5enl8736h/post/191>)

Compared with sdg, the advantage of Adam is the Adam algorithm uses an adaptive learning rate, which can dynamically adjust the learning rate according to the gradient of each parameter, so that the learning rate of different parameters is different and the adaptability is stronger.

In addition, the construction of the model is similar to the previous method, see Task(c)'s answer for details. The difference from before is that because of the addition of a loss this time, more training adjustments can lead to a better model. This time I use 20 model adjustments.

## b.2 Result visualization

The correct log file is stored in the same directory as log\_file\_3.txt.

The tables 2.3(b) 1 and table 2.3(b) 2 show the value of the accuracy of 20 trainings on the sketch test.

Fig 2.2(b) 1 is the screen shot of my code result, the Fig 2.2(b) 2 is the line graph of 20 epoch accuracy.

| Epoch    | 1      | 2      | 3      | 4      | 5      | 6      | 7      | 8      | 9      | 10     |
|----------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|
| accuracy | 50.89% | 56.72% | 58.03% | 60.52% | 56.12% | 56.96% | 56.36% | 56.60% | 61.47% | 57.79% |

table 2.3(b) 1

| Epoch    | 11     | 12     | 13     | 14     | 15     | 16     | 17     | 18     | 19     | 20     |
|----------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|
| accuracy | 62.43% | 58.62% | 60.76% | 48.51% | 56.12% | 53.39% | 61.83% | 61.47% | 59.22% | 63.97% |

table 2.3(b) 2

Transfer learning: Epoch 1 - Sketch Test Accuracy on real: 50.89%  
 Transfer learning: Epoch 2 - Sketch Test Accuracy on real: 56.72%  
 Transfer learning: Epoch 3 - Sketch Test Accuracy on real: 58.03%  
 Transfer learning: Epoch 4 - Sketch Test Accuracy on real: 60.52%  
 Transfer learning: Epoch 5 - Sketch Test Accuracy on real: 56.12%  
 Transfer learning: Epoch 6 - Sketch Test Accuracy on real: 56.96%  
 Transfer learning: Epoch 7 - Sketch Test Accuracy on real: 56.36%  
 Transfer learning: Epoch 8 - Sketch Test Accuracy on real: 56.60%  
 Transfer learning: Epoch 9 - Sketch Test Accuracy on real: 61.47%  
 Transfer learning: Epoch 10 - Sketch Test Accuracy on real: 57.79%  
 Transfer learning: Epoch 11 - Sketch Test Accuracy on real: 62.43%  
 Transfer learning: Epoch 12 - Sketch Test Accuracy on real: 58.62%  
 Transfer learning: Epoch 13 - Sketch Test Accuracy on real: 60.76%  
 Transfer learning: Epoch 14 - Sketch Test Accuracy on real: 48.51%  
 Transfer learning: Epoch 15 - Sketch Test Accuracy on real: 56.12%  
 Transfer learning: Epoch 16 - Sketch Test Accuracy on real: 53.39%  
 Transfer learning: Epoch 17 - Sketch Test Accuracy on real: 61.83%  
 Transfer learning: Epoch 18 - Sketch Test Accuracy on real: 61.47%  
 Transfer learning: Epoch 19 - Sketch Test Accuracy on real: 59.22%  
 Transfer learning: Epoch 20 - Sketch Test Accuracy on real: 63.97%

Fig 2.2(b) 1

Using migration learning to train on the real data set, the accuracy obtained on the sketch data set

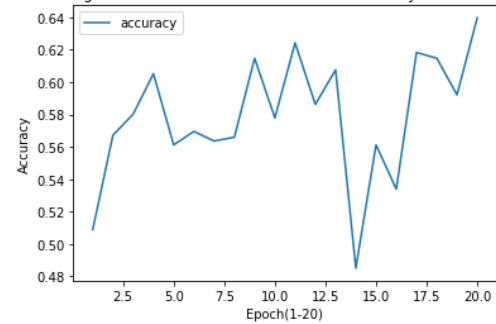


Fig 2.2(b) 2

## b.3 Observation and analysis

Calculate the average of 10 correct rates as follow,

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = 57.89\%$$

And the variance,

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 = 14.30\%$$

Among them,  $n$  represents the number of samples,  $x_i$  represents the  $i$ th sample,  $\bar{x}$  represents the sample mean, and  $s^2$  represents the sample variance.

The average of the 20 epoch shows that the accuracy recognition has reached more than 50%. The large variance indicates that the performance of the model has changed a lot in one iteration

The test set accuracy of the model varies greatly in 20 optimizations, ranging from 48.51% to 63.97%. Some optimizations will reduce the performance. For example, after the 14th optimization, the accuracy the test set drops to 48.51%. In addition, the accuracy the test set shows an overall upward trend, and the highest correct rate is 63.97%, which is reached at the 20th optimization. This may indicate that the model is constantly learning and improving.

#### b.4 Compare With 2.2(c)

Result in 2.2(c):

| Epoch    | 1      | 2      | 3      | 4      | 5      | 6      | 7      | 8      | 9      | 10     |
|----------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|
| accuracy | 47.32% | 46.37% | 46.61% | 42.57% | 50.77% | 46.49% | 42.93% | 46.37% | 49.70% | 48.63% |

table 2(c) 3

Result in 2.3(b)

| Epoch    | 1      | 2      | 3      | 4      | 5      | 6      | 7      | 8      | 9      | 10     |
|----------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|
| accuracy | 50.89% | 56.72% | 58.03% | 60.52% | 56.12% | 56.96% | 56.36% | 56.60% | 61.47% | 57.79% |

table 2.3(b) 3

| Epoch    | 11     | 12     | 13     | 14     | 15     | 16     | 17     | 18     | 19     | 20     |
|----------|--------|--------|--------|--------|--------|--------|--------|--------|--------|--------|
| accuracy | 62.43% | 58.62% | 60.76% | 48.51% | 56.12% | 53.39% | 61.83% | 61.47% | 59.22% | 63.97% |

table 2.3(b) 4

For the first set of data (without domain adaptation), the average accuracy is 46.33%, with a minimum of 42.57% and a maximum of 50.77%. The range is relatively small, which means that the optimization obtained after multiple model adjustments is not obvious. Compared to the second set of data (with domain adaptation), the average accuracy is lower, indicating that the model has difficulty generalizing to the test data.

For the second set of data (domain adaptation), the average accuracy is 57.89%, the lowest is 48.51%, and the highest is 63.97%. This range is larger than the first set of data, indicating that the performance of the model is optimized after tuning using domain adaptation. The average accuracy is significantly higher than the first set of data, indicating that domain adaptation improves the generalization performance of the model on the test data.

#### T-test

In order to prove that the model after domain adaptation can get better results, we use the T test method to compare the performance of the two models.

We refer to the correct rate from the 2.2(c) model as data1. The correct rate from 2.3(b) becomes data2.

First, we identify the null and alternative hypotheses for the hypothesis test. In this example, the null hypothesis can be set to "the means of the two sets of data are equal" and the alternative hypothesis can be set to "the means of the two sets of data are not equal".

Since data1 and data2 are from two independent data sets, we use independent sample t test.

After executing `ttest_ind(data1, data2, equal_var=False)` in python, the p value is 0.00023538494703963194 less than 0.05.

At the 95% confidence level, we reject the null hypothesis and accept the alternative hypothesis that the means of the two sets of data are not equal. Therefore, combining the results of the t test and the values of data1 and data2, we can conclude that the model of 2.3(b) is better.

c. (1 )In the previous question, we used the recommended weight for the domain adaptation loss function. Does this recommended weight always work in practice?

Please explain your answer.

The method I used in the previous question is the domain adaptation method of MMD loss. The recommended weight for domain adaptation is  $\text{lambda} = 0.01$ .

In order to see the impact of the recommended value of  $\text{lambda} = 0.01$  on the model, I added the calculation of  $\text{lambda} * \text{mmd\_loss}(\text{feature loss})$ , source loss and total loss to the `train_transfer` function (explained in detail in Task 2.3a). And encapsulate them in the list. Then visualize the three losses and print them all in the same graph to observe their trend changes.

### Result visualization



### observation and analysis

It can be seen from the above figure that after each epoch, `src_loss` and `total_loss` show a gradual downward trend. In addition, the value of `lambda*mmd_loss` is very small, close to 0, so that `src_loss` and `total_loss` almost coincide on the image. It can be seen from `total_loss = src_loss + lambda*mmd_loss` that the `total_loss` value of the model is mainly affected by `src_loss`.

According to the principle of the `train_transfer` function, we know that the function backpropagates the model according to the `total_loss` and updates the model parameters.

```
1. loss.backward()
2. # Update model parameters using the optimizer
3. optimizer.step()
```

However, because the `lambda` value recommended by the title is too small, the influence of `mmd_loss` on model optimization becomes weak. This may lead to insufficient generalization ability of the model, making it difficult to cope with the recognition challenges posed by different domains.

Therefore, in practice, using the recommended weights ( $\text{lambda} = 0.001$ ) of the domain adaptation loss function does not always work. This is because different tasks and domains may have different distribution characteristics and complexities.

In order to improve the generalization ability of the model and better adapt to different fields, we need to try to adjust the value of `lambda` so that `mmd_loss` can play a greater role in model optimization. This can be achieved through experimentation and validation to find

the optimal combination of weights for a particular problem and domain.

**(2) Our textbook, on the other hand, assumes no domain gap between training and testing and recommends using a labelled validation set for hyperparameters. In practical scenarios, where we have a labelled source domain and an unlabelled target domain with distribution shifts, what will be the problem of the textbook way of selecting hyperparameters? Please provide necessary explanations.**

Regarding the textbook approach to hyperparameter selection, using a labeled validation set can lead to poor performance of the model on the target domain in the presence of a domain gap between training and testing. This is because the labels of the verification set come from the source domain, so the data distribution of the verification set is the same as that of the training set, while the data distribution of the target domain may be significantly different from the source domain, and there may be a domain gap between the target domain. Therefore, in this case, tuning the hyperparameters using the labeled validation set of the source domain may not adequately fit the data distribution of the target domain, resulting in reduced generalization performance of the model on the target domain.

For example, suppose we are training a deep learning model for image classification. The source domain contains many images with class labels, while the target domain contains unlabeled images with a shifted distribution. When using a textbook approach to hyperparameter selection, we may achieve high accuracy on the source domain, but perform poorly on the target domain because the data distribution of the target domain and the source domain may be different. This is because the model may fail to capture key features and patterns in the target domain, causing its generalization performance on the target domain to suffer.